[SoC Design]

Week 6

(from github repo)

Report 6

Student Number: 2023066980

Student Name: 김영진

Explain the waveform results along with the implemented "to-do" code, including the operation of the testbench.

- **Include the captured "to-do" code.**
- **Include the captured waveform.**
- **Write the report in English.**

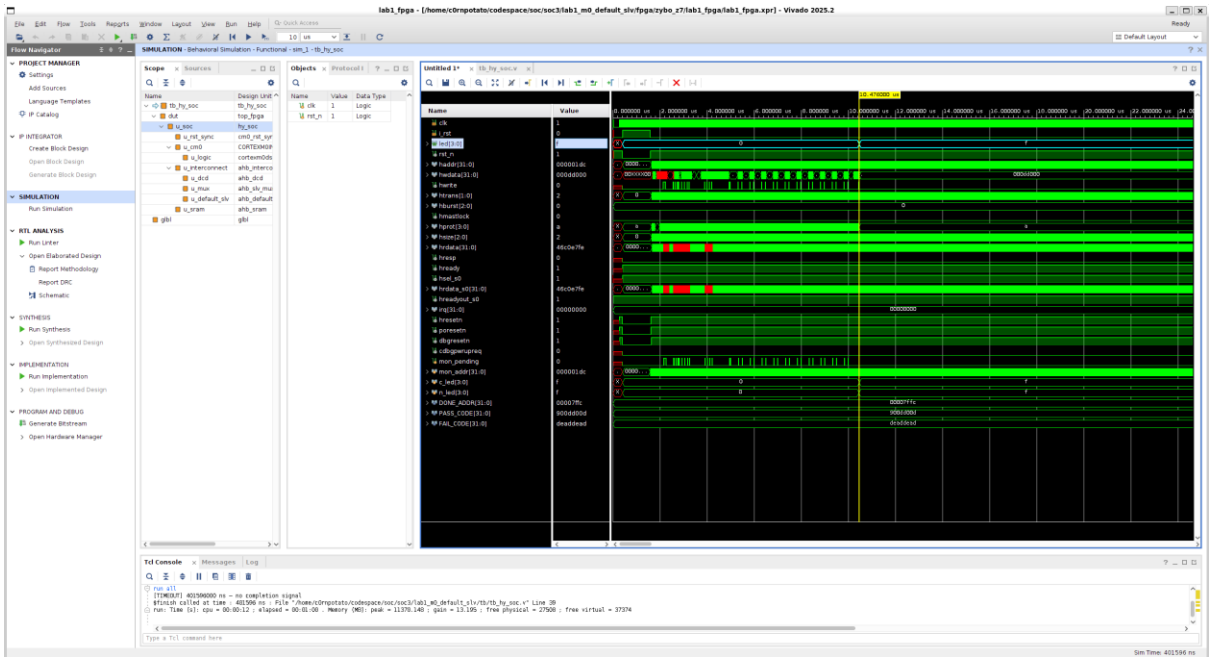
Lab1

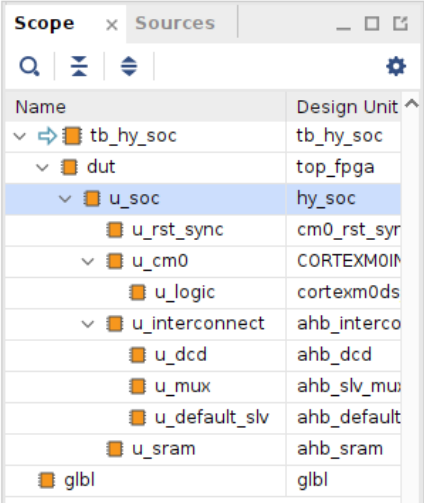
a) **Simulation with the pre-compiled code (Capture Full Screen)**

b) **Simulation Passed Code MDK Keil + test.c code (Capture Full Screen, Only one)**

c) **FPGA Board Demo**

a)

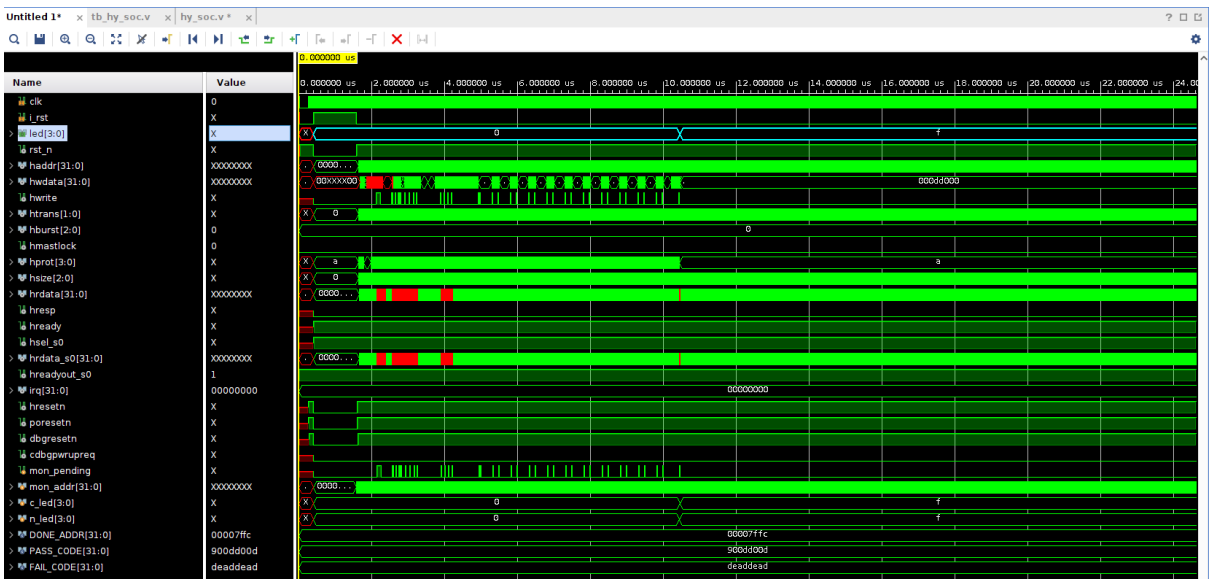




Untitled 1* | tb_hy_soc.v | hy_soc.v *

/home/c0rnpotato/codespace/soc/soc3/lab1_m0_default_slv/rtl/hy_soc.v

```
208
209     // combinational keyword = {DONE_ADDR, PASS_CODE, FAIL_CODE, hwdata, mon_addr, mon_pending}
210     always @(*) begin
211       n_led = c_led; // default (□□)
212
213       if ( mon_pending && mon_addr == DONE_ADDR ) begin
214         if ( hwdata == PASS_CODE )
215           n_led = 4'b1111;
216         else if ( hwdata == FAIL_CODE )
217           n_led = 4'b0101;
218       end
219     end
220
221     assign led = c_led;
222   endmodule
223
```



```
// --==========================================================================--
// Copyright (c) 2025-2026 DSAL, Hanyang University. All rights reserved
//                      DSAL Confidential Proprietary
//  ---------------------------------------------------------------------------
//  This confidential and proprietary software may be used only as
//  authorised by a licensing agreement from DSAL.
// ---------------------------------------------------------------------------
// FILE NAME       : hy_soc.v
// DEPARTMENT      : Digital System Architecture Lab.
// AUTHOR          : Ji-Hoon Kim
// AUTHOR'S EMAIL  : jhoonkim@hanyang.ac.kr
// ---------------------------------------------------------------------------
// hy_soc.v - HY-SoC Educational Lab1
//
// Cortex-M0 (DesignStart) + AHB-Lite Shared Bus + SRAM 64KB + Default Slave
//
// Memory map:
//   0x0000_0000 - 0x00FF_FFFF : s0 - ahb_sram (64 KB actual, 16 MB window)
//   other                     : nomap → ahb_default_slv (ERROR response)
module hy_soc (
  input wire clk,     // system clock
  //input wire rst_n,    // async assert, sync deassert (active-low)
  input wire i_rst,
  output wire [3:0] led
);
wire rst_n;
assign rst_n = !i_rst;
  // ---------------------------------------------------------------------------
  // AHB-Lite bus signals
  // ---------------------------------------------------------------------------
  wire [31:0] haddr;
  wire [31:0] hwdata;
  wire        hwrite;
  wire [1:0]  htrans;
  wire [2:0]  hburst;
  wire        hmastlock;
  wire [3:0]  hprot;
  wire [2:0]  hsize;
  wire [31:0] hrdata;
  wire        hresp;
  wire        hready;
  // Subordinate signals
  wire        hsel_s0;
  wire [31:0] hrdata_s0;
  wire        hreadyout_s0;
  // No interrupts in v02
  wire [31:0] irq = 32'h0;
  // ---------------------------------------------------------------------------
  // Reset synchronizer: two independent async-assert / sync-deassert chains
  //   por_sync (ext reset only)        -> PORESETn, DBGRESETn
  //   sys_sync (ext reset + SYSRESETREQ) -> HRESETn
  // ---------------------------------------------------------------------------
  wire hresetn, poresetn, dbgresetn;
  cm0_rst_sync u_rst_sync (
    .clk            (clk),
```

```
  .rst_n          (rst_n),
  .sysresetreq_i  (1'b0),
  .poresetn_o     (poresetn),
  .dbgresetn_o    (dbgresetn),
  .hresetn_o      (hresetn)
);
// CDBGPWRUP self-ack
wire cdbgpwrupreq;
// ----------------------------------------------------------------------
// Cortex-M0 DesignStart
// ----------------------------------------------------------------------
CORTEXM0INTEGRATION u_cm0 (
  // clocks & resets
  .FCLK           (clk),
  .SCLK           (clk),
  .HCLK           (clk),
  .DCLK           (clk),
  .PORESETn       (poresetn),
  .DBGRESETn      (dbgresetn),
  .HRESETn        (hresetn),
  .SWCLKTCK       (1'b0),
  .nTRST          (1'b1),
  // AHB-Lite manager port
  .HADDR          (haddr),
  .HBURST         (hburst),
  .HMASTLOCK      (hmastlock),
  .HPROT          (hprot),
  .HSIZE          (hsize),
  .HTRANS         (htrans),
  .HWDATA         (hwdata),
  .HWRITE         (hwrite),
  .HRDATA         (hrdata),
  .HREADY         (hready),
  .HRESP          (hresp),
  .HMASTER        (),
  // code sequentiality (unused)
  .CODENSEQ       (),
  .CODEHINTDE     (),
  .SPECHTRANS     (),
  // debug - SWD tied off
  .SWDITMS        (1'b0),
  .TDI            (1'b0),
  .SWDO           (),
  .SWDOEN         (),
  .TDO            (),
  .nTDOEN         (),
  .DBGRESTART     (1'b0),
  .DBGRESTARTED   (),
  .EDBGRQ         (1'b0),
  .HALTED         (),
  // interrupts
  .NMI            (1'b0),
  .IRQ            (irq),
  .TXEV           (),
  .RXEV           (1'b0),
  .LOCKUP         (),
  .SYSRESETREQ    (),
  // SysTick: 50 MHz → 10 ms = 500 000 - 1 = 0x07_A11F
  .STCALIB        ({1'b1, 1'b0, 24'h07A11F}),
```

```
  .STCLKEN        (1'b0),
  .IRQLATENCY     (8'h00),
  .ECOREVNUM      (28'h0),
  // power management - tied off
  .GATEHCLK       (),
  .SLEEPING       (),
  .SLEEPDEEP      (),
  .WAKEUP         (),
  .WICSENSE       (),
  .SLEEPHOLDREQn  (1'b1),
  .SLEEPHOLDACKn  (),
  .WICENREQ       (1'b0),
  .WICENACK       (),
  .CDBGPWRUPREQ   (cdbgpwrupreq),
  .CDBGPWRUPACK   (cdbgpwrupreq),  // self-ack
  // scan - tied off
  .SE             (1'b0),
  .RSTBYPASS      (1'b0)
);
// ---------------------------------------------------------------------------
// On-chip Interconnect (SRAM + default slave)
// ---------------------------------------------------------------------------
ahb_interconnect u_interconnect (
  .hclk            (clk),
  .hrst_n          (hresetn),
  .haddr_i         (haddr),
  .htrans_i        (htrans),
  .hrdata_o        (hrdata),
  .hready_o        (hready),
  .hresp_o         (hresp),
  .hsel_sram_o     (hsel_s0),
  .hrdata_sram_i   (hrdata_s0),
  .hreadyout_sram_i(hreadyout_s0)
);
// ---------------------------------------------------------------------------
// s0: ahb_sram 64 KB (MEMWIDTH=16 → 2^16 bytes)
// ---------------------------------------------------------------------------
ahb_sram #(.MEMWIDTH(16)) u_sram (
  .hsel      (hsel_s0),
  .clk       (clk),
  .rst_n     (hresetn),
  .hready    (hready),
  .haddr     (haddr),
  .htrans    (htrans),
  .hwrite    (hwrite),
  .hsize     (hsize),
  .hwdata    (hwdata),
  .hreadyout (hreadyout_s0),
  .hrdata    (hrdata_s0)
);

localparam DONE_ADDR = 32'h00007FFC;
localparam PASS_CODE = 32'h900DD00D;
localparam FAIL_CODE = 32'hDEADDEAD;

reg        mon_pending;
reg [31:0] mon_addr;
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
```

```
        mon_pending <= 1'b0;
        mon_addr    <= 32'h0;
    end else if (hready) begin
        mon_pending <= hsel_s0 && hwrite && htrans[1];
        mon_addr    <= haddr;
    end
  end

  // state
  reg [3:0] c_led, n_led;
  // sequential
  always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
      c_led <= 4'b0000;
    else
      c_led <= n_led;
  end
  // combinational keyword = {DONE_ADDR, PASS_CODE, FAIL_CODE, hwdata, mon_addr, mon_pending}
  always @(*) begin
    n_led = c_led; // default (중요)
    if ( mon_pending && mon_addr == DONE_ADDR ) begin
      if ( hwdata == PASS_CODE )
        n_led = 4'b1111;
      else if ( hwdata == FAIL_CODE )
        n_led = 4'b0101;
    end
  end
  assign led = c_led;
endmodule
```

b)

D:\hanyang\4_1\SoC\week5\week5.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Project

- Project: week5
  - Target_1
    - Source Group 1
      - core_cm0.h
      - core_cmFunc.h
      - core_cmInstr.h
      - startup_cm0.S
      - test.c
      - hy_soc.h

startup_cm0.S | test.c | hy_soc.h

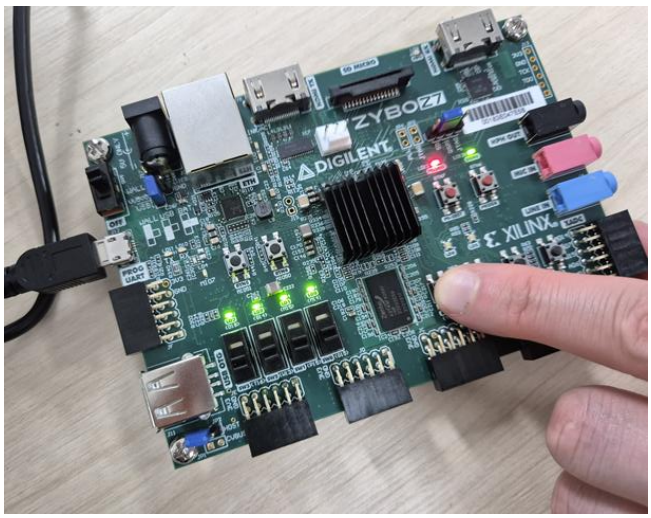
```
 * No SysTick or named exception handlers needed.
 */
    .syntax unified
    .cpu cortex-m0
    .thumb
    .eabi_attribute 25, 1
/* ---------- Stack top (matches scatter.sct ARM_LIB_STACK) ---------- */
    .equ    __initial_sp, 0x00010000    /* SRAM 64 KB top */
    .globl  __initial_sp

/* ---------- Vector Table (RESET area) ---------- */
    .section RESET, "a", %progbits
    .p2align 3
    .globl  __Vectors
    .globl  __Vectors_End
    .globl  __Vectors_Size

__Vectors:
    .word   __initial_sp        /* 0x00: Initial Stack Pointer          */
    .word   Reset_Handler       /* 0x04: Reset                          */
    .word   Default_Handler     /* 0x08: NMI                            */
    .word   Default_Handler     /* 0x0C: Hard Fault                     */
    .rept   12
    .word   Default_Handler     /* 0x10-0x3C: reserved + SysTick        */
    .endr
    .rept   16
    .word   Default_Handler     /* 0x40-0x7C: IRQ0-15                   */
    .endr
__Vectors_End:

    .equ __Vectors_Size, __Vectors_End - __Vectors

/* ---------- Reset Handler ---------- */
    .section .text
    .p2align 2
    .thumb_func
    .globl  Reset_Handler
    .type   Reset_Handler, %function
Reset_Handler:
    ldr     r0, =__main         /* Arm C library: scatter-load + __rt_entry → main() */
    bx      r0

/* ---------- Default Handler (weak) ---------- */
    .thumb_func
    .weak   Default_Handler
    .type   Default_Handler, %function
Default_Handler:
    b       .

    .end
```

Project | Books | Functions | Templates

Build Output

```
Rebuild started: Project: week5
*** Using Compiler 'V6.24', folder: 'E:\MDK\ARM\ARMCLANG\Bin'
Rebuild target 'Target_1'
Note: source file '.\hy-soc_lab\edu\lab0_m0_sram\sw\arm\startup_cm0.S' - object file renamed from '.\Objects\startup_cm0.o' to '.\Objects\startup_cm0_1.o'.
Note: source file '.\hy-soc_lab\edu\lab0_m0_sram\sw\arm\startup_cm0.S' - object file renamed from '.\Objects\startup_cm0.o' to '.\Objects\startup_cm0_1.o'.
compiling test.c...
assembling startup_cm0.S...
linking...
Program Size: Code=372 RO-data=128 RW-data=0 ZI-data=2048
After Build - User command #1: fromelf --vhx --32x1 --output=code.hex ./Objects/week5.axf
".\Objects\week5.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:01
```

c)



1. Warning Resolution

The eabi_attribute 25, 1 directive was added to startup_cm0.S to suppress existing build warnings.

2. Monitor Logic Description

The monitor logic is designed to observe write access to the **ahb_sram**. It triggers specifically when the target address matches the debugging address, defined as DONE_ADDR.

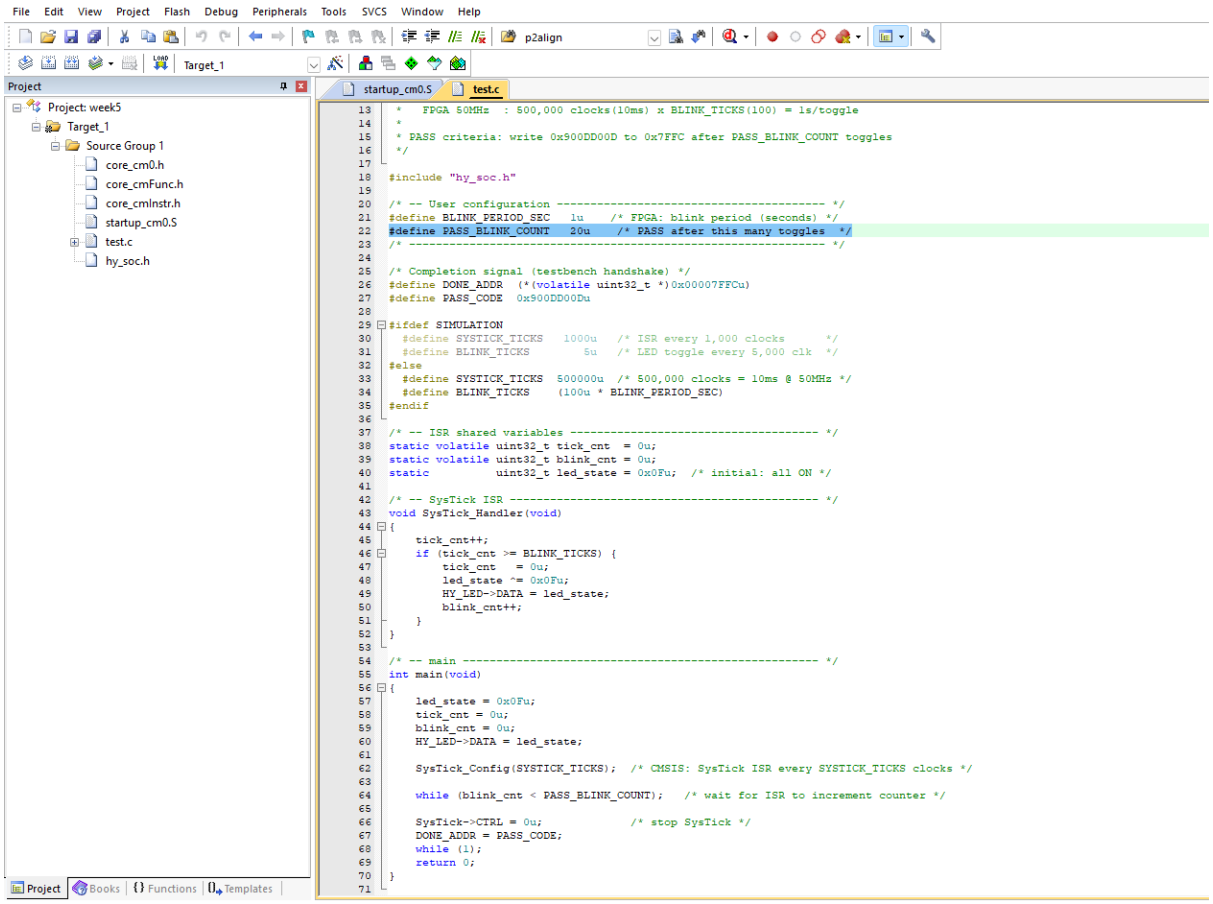
Specifically, the logic does not operate by performing a read access to DONE_ADDR. Instead, it monitors the **write phase**; at the exact moment a write operation is asserted to DONE_ADDR, the logic inspects the hwdata to determine the **PASS** or **FAIL** status of the simulation.

3. Comparison with Week 5 Lab

In the previous week's RTL implementation, only the SRAM was connected to the AHB bus as a subordinate. The design was configured to perform write operations unconditionally, without accounting for the master’s specific request states.

However, in the current revision, the **interconnect** has been modified to manage the ready signal and control write access to the slave channels. This architecture enhances system stability by arbitrating signals such as hready and hdata through the slv_mux module. While Lab 1 continues to utilize a single slave, this implementation is critical for preventing **race conditions** in Lab 2 and beyond, where multiple slaves are involved.

Lab2



1. Build Optimization

Similar to the previous update, build warnings were suppressed by adding the eabi_attribute 25, 1 directive to startup_cm0.S.

2. LED Control Logic and Configuration

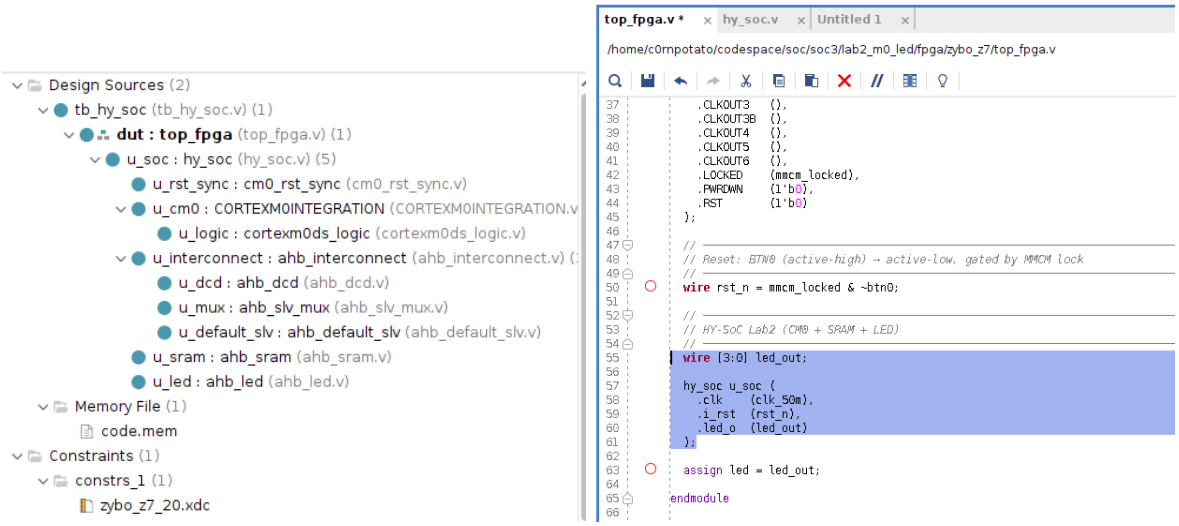
In Test.c, the PASS_BLINK_COUNT was adjusted to **20**.

- **Blink Definition**: For this implementation, a single "blink" is defined as a complete **On → Off → On** cycle.

- **Alternative Definition**: If a "blink" were defined as a single state transition (either On → Off or Off → On), a value of **10** would be appropriate.

3. ISR Implementation Details

This configuration is dictated by the led_state ^= 0x0Fu; statement within the **SysTick ISR**. Since the bitwise **XOR (^) operation** is executed every time the interrupt handler is invoked, it toggles the 4-bit LED state between its active and inactive phases. The total number of these inversions is controlled by the PASS_BLINK_COUNT value, ensuring the desired visual feedback duration.



In addition, the interconnect architecture has been updated relative to the earlier version of the code.

```
module ahb_slv_mux (…
);

  localparam MUX_SRAM  = 2'd0;
  localparam MUX_LED   = 2'd1;
  localparam MUX_NOMAP = 2'd2;

  reg [1:0] mux_sel;

  always @(posedge hclk or negedge hrst_n)
    if (!hrst_n)       mux_sel <= MUX_NOMAP;
    else if (hready_i) begin
      if      (hsel_sram_i) mux_sel <= MUX_SRAM;
      else if (hsel_led_i)  mux_sel <= MUX_LED;
      else                  mux_sel <= MUX_NOMAP;
    end

  reg [31:0] hrdata_mux;
  reg        hready_mux;
  reg        hresp_mux;

  always @* begin
    case (mux_sel)
      MUX_SRAM: begin hrdata_mux = hrdata_sram_i; hready_mux = hreadyout_sram_i; hresp_mux = 1'b0; end
      MUX_LED:  begin hrdata_mux = hrdata_led_i;  hready_mux = hreadyout_led_i;  hresp_mux = 1'b0; end
      default:  begin hrdata_mux = hrdata_nomap_i; hready_mux = hreadyout_nomap_i; hresp_mux = hresp_nomap_i; end
    endcase
  end

  assign hrdata_o = hrdata_mux;
  assign hready_o = hready_mux;
  assign hresp_o  = hresp_mux;

endmodule
```

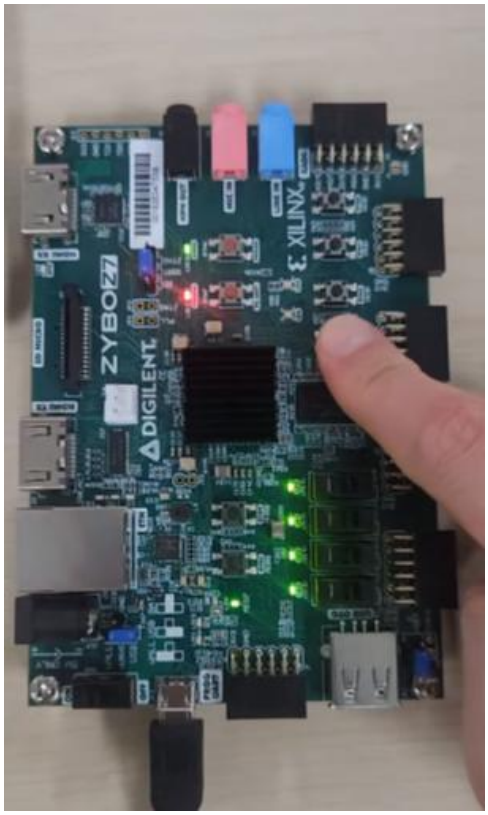
```
module ahb_dcd (
  input  wire [31:0] haddr_i,

  output wire        hsel_sram_o,
  output wire        hsel_led_o,
  output wire        hsel_nomap_o
);

  assign hsel_sram_o  = (haddr_i[31:16] == 16'h0000);
  assign hsel_led_o   = (haddr_i[31:24] == 8'h50);
  assign hsel_nomap_o = ~(hsel_sram_o | hsel_led_o);

endmodule
```

1. Interconnect and Arbitration Logic

In the previous implementation, the logic was limited to supporting only the SRAM. However, the addition of multiple slaves necessitated an arbitration mechanism to manage channel access rights. Consequently, **multiplexing (MUX) logic** was integrated to handle the data paths for both the **LED** and **SRAM** subordinates.

2. Implementation and Debugging

During the **instantiation** phase, a discrepancy in the pin naming convention was identified and corrected to ensure proper signal mapping.

3. Verification Strategy

The firmware developed in Keil is configured with a **1-second blink interval**. Since simulating such a long period at the RTL/Gate level is computationally expensive and impractical, the functional verification was instead performed through **FPGA prototyping**.